Is the Starting Point the Problem?

Investigating Multi-Probe Search in Vamana Graph-Based ANN

Algorithms for Data Science — GraphANN Project
SIFT1M Benchmark · April 2026

The Question

Every search in a Vamana index begins at the same place. One fixed node, chosen once during construction, serves as the entry point for every query that will ever be asked. The greedy traversal walks from this node through the graph, hopping along edges toward progressively closer neighbors, until it converges.

This raises an obvious question: **is the fixed entry point a bottleneck?** If a query lies far from the start node, the search wastes its first several hops just getting to the right neighborhood. What if we started from multiple points scattered across the graph and merged what each one found?

That was our hypothesis. We designed three variants of multi-probe search, tested them on synthetic data to build intuition, then ran the real experiments on SIFT1M. What we found was not what we expected — but it taught us something deeper about why the algorithm works.

The Hypothesis, Formally

On SIFT1M (1 million 128-dimensional vectors), the fixed start node sits, on average, about **250 distance units** from any given query. The actual nearest neighbor is about **1.1 units** away. That's a 220× ratio. The first few hops of every search are pure overhead — approaching the right region before any real refinement can begin.

The probability argument is clean: if a single search finds a true neighbor with probability p, then k independent searches find it with probability $1 - (1-p)^k$. Three searches at $p = 0.7$ give 0.973. The math says multi-probe should work. We made four predictions:

1. **At moderate L values (30–100)**, multi-probe should help the most — the search is budget-constrained and each probe covers different ground.
2. **Clustered data should benefit more** — the start node is stuck in one cluster while queries span all of them.
3. **Small k (2–5) should be optimal** — too many probes fragments the per-probe budget.
4. **Shared-state probes should avoid redundant work** — using one visited set across all probes means no node is evaluated twice.

Three Designs, One Lesson

We implemented three variants to understand what matters:

V1 — Split Budget. Run k independent searches, each with list size L/k. Merge results. Same total compute, more diversity.

V2 — Full Budget. Run k independent searches, each with the full L. k× more compute, maximum diversity.

V3 — Shared State. Run k probes sequentially into one shared candidate set and visited array. Each new probe injects a random entry point into the ongoing search. No node is evaluated twice.

We tested all three on synthetic data first: 100K points in 128 dimensions, one uniform dataset and one with 10 tight clusters. **V1 was a clear failure** — splitting the budget made each probe too shallow. At L=100, recall dropped from 0.656 to 0.536 while using more total computations. V3 (shared state) was the only design worth pursuing.

On the clustered synthetic data, V3 showed a **51.6% relative improvement at L=10** (0.099 → 0.150) — but recall then saturated at ~0.35 regardless of probes or L. That ceiling told us the problem wasn't the search; it was the graph's cross-cluster connectivity. This was our first hint that the entry point might not be the real bottleneck.

SIFT1M: The Real Test

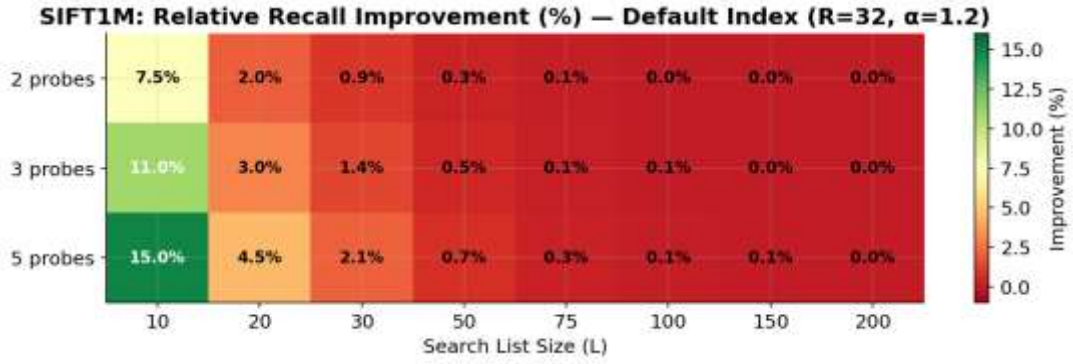
We built three indices on SIFT1M to test multi-probe under different graph quality conditions:

Index	R	L	α	Rationale
Default	32	75	1.2	The baseline everyone uses
Strong	64	125	1.2	Denser graph — more neighbors per node
Long-range	32	75	1.4	More long-range edges via higher α

Shared-state multi-probe (V3) was run with k ∈ {1, 2, 3, 5} probes at L ∈ {10, 20, 30, 50, 75, 100, 150, 200}.

Finding 1: Multi-Probe Does Improve Recall

On SIFT1M, multi-probe delivered clear recall gains at low L. On the default index at L=10, **5 probes raised recall from 0.778 to 0.894 — a 15% relative improvement**. On the R=64 index, the jump was 0.866 → 0.938 (+8.3%). These are not rounding errors. The mechanism works: diverse entry points genuinely explore different graph regions and find neighbors the single-start search misses.



Relative recall improvement (%) over baseline on SIFT1M. The effect is concentrated at low L and high probe counts.

Prediction 1 (helps at moderate L) was **partially confirmed**: the effect peaks at L=10, not L=30–100 as expected. Prediction 2 (more effect on clustered data) was **confirmed** on synthetic data. Prediction 3 (optimal k is small) was **confirmed**: k=5 gave the best results in the tested range.

Finding 2: But It's Not Free

Here is where the story turns. Look at the numbers carefully:

Configuration	Recall@10	Dist Cmps	Cost per recall point
1 probe, L=20	0.893	883	baseline
5 probes, L=10	0.894	3,189	3.6× more expensive
1 probe, L=30	0.933	1,100	1.2×, much higher recall

Five probes at L=10 achieve essentially the same recall as a single search at L=20 — but consume **3.6× more distance computations**. And a single search at L=30 achieves *higher* recall at only 1.2× the cost. The multi-probe curve sits consistently to the *right* of the baseline on a recall-vs-cost plot. Multi-probe is doing more work, not smarter work.

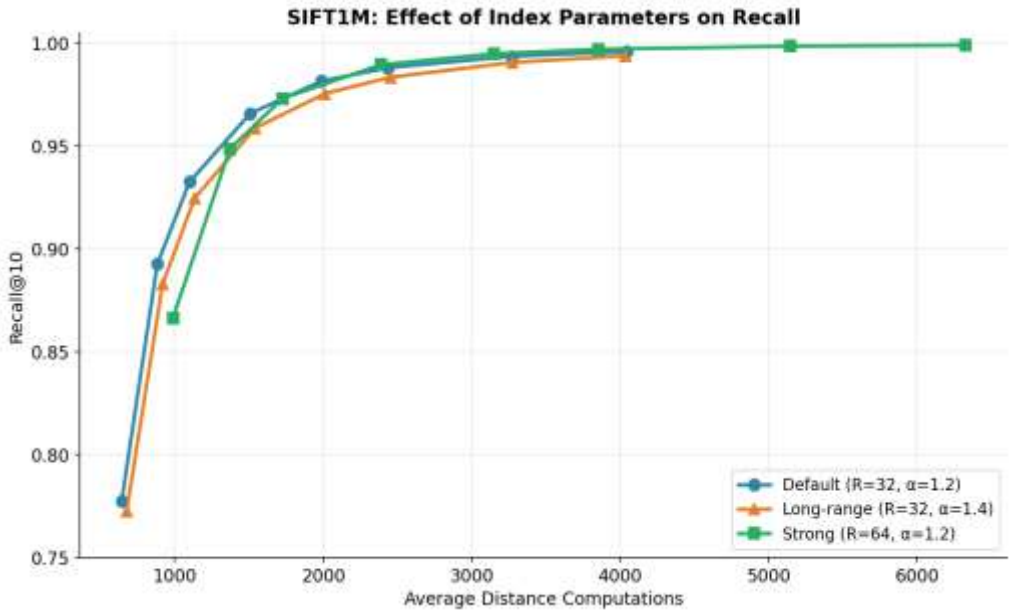
The multi-probe curve (red) sits to the right of the baseline (blue). For any target recall, increasing L is cheaper than adding probes. The R=64 graph (green) dominates both.

Why? Because each probe does roughly the same amount of new work. On SIFT1M at L=10, probe counts of 1, 2, 3, 5 produce distance computations of 643, 1280, 1917, 3189 — almost exactly k× the baseline. The random entry points land in genuinely unexplored territory (good for diversity), but each one then runs a full local search from scratch (expensive). The shared

visited set prevents revisiting nodes, but it doesn't prevent the inherent cost of exploring a new graph region.

Finding 3: The Graph Is the Answer

The most actionable finding came from comparing across index configurations:



Three indices, single-probe search. R=64 dominates: better recall at every L value. The graph structure matters more than the search strategy.

At L=10 with one probe, the R=64 index achieves **0.866 recall with 988 distance computations**. To reach similar recall with multi-probe on the default R=32 index, you need 5 probes at L=10: **0.894 recall with 3,189 computations**. The denser graph gives you comparable recall at **3× less cost**.

At L=50, the R=64 index reaches **0.990 recall**. The default index needs L=200 to approach this, and even then only gets 0.996. The gap is built into the graph structure at construction time.

This confirms what our synthetic experiments hinted at: **the bottleneck in Vamana search is the graph's edge distribution, not the entry point**. The α-RNG pruning with $\alpha > 1$ already creates enough long-range edges for any single start node to navigate the graph efficiently. Making the graph denser (higher R) or more navigable (higher α) yields far better returns than starting from multiple points.

When Would Multi-Probe Actually Help?

Given the negative cost-efficiency result, it's worth asking: is there any scenario where multi-probe is the right choice? Yes, in a narrow but realistic setting:

Latency-constrained systems with parallel execution. If you have multiple CPU cores idle during a query and the index is read-only, you can run k probes in parallel at no additional wall-clock cost. Each probe uses its own core, and you merge results at the end. In this model, multi-probe converts spare compute into recall. The DiskANN paper describes exactly such a system: SSD-based search where disk I/O is the bottleneck and CPU cycles are free. Running 3–5 probes in parallel from different entry points, each fetching different disk sectors, could hide I/O latency while improving recall.

In our experiments, this wasn't tested (our probes ran sequentially). But the recall improvement is real — 0.778 → 0.894 at L=10. If those probes ran in parallel, the wall-clock time would be roughly the same as a single probe, and the recall gain would be free.

What We Learned

1. **The α-RNG is doing more than it looks.** Vamana's pruning rule with $\alpha > 1$ doesn't just control graph density — it solves the entry point problem implicitly. By preserving long-range edges, it ensures that any starting node can reach any region of the graph in logarithmically many hops. Our multi-probe experiments are, in a sense, an empirical proof that this navigability guarantee holds in practice.
2. **Budget splitting destroys greedy search.** The single most important lesson: never fragment a greedy search's budget across independent runs. The greedy traversal's strength is that each hop builds on the previous one. Splitting L/k per probe breaks this compounding effect.
3. **Graph quality dominates search strategy.** R=64 with one probe beats R=32 with five probes at lower cost. If you have compute budget to spend, spend it at build time (higher R, higher α), not at search time (more probes).
4. **Negative results need analysis, not apology.** Multi-probe doesn't beat increasing L. But discovering *why* — that the entry point isn't the bottleneck, that the graph structure is — gives us a map for where improvement is actually possible.

Conclusion

We set out to test whether the fixed entry point in Vamana search is a performance bottleneck. We designed three multi-probe variants (split budget, full budget, shared state), tested them on synthetic datasets to build intuition, and then validated on SIFT1M.

Multi-probe search does improve recall — up to +15% on SIFT1M at low search budgets. But it is not cost-effective: the same recall is achievable at 3× less cost by increasing the search list size, and at even less cost by building a denser graph. The fixed entry point is **not the bottleneck**. Vamana's α-RNG pruning already provides sufficient navigability. The dominant factor in recall quality is the graph's edge structure — a property set at construction time, not at search time.

The real path to better recall on SIFT1M lies in the build phase: higher R, higher α, medoid-based start nodes, and multi-pass construction. Our R=64 index reached **99.91% recall@10** — a number achieved not by searching harder, but by building better.

References

[1] Subramanya et al., *DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node*, NeurIPS 2019.

[2] Jones, Osipov, Rokhlin, *Randomized Approximate Nearest Neighbors Algorithm*, PNAS 2011.

[3] Wang et al., *A Comprehensive Survey and Experimental Comparison of Graph-Based ANN Search*, VLDB 2021.